

Tenbin Audit Scope 4/14/26

Commit Hash

[820e700943c2b657c54219ab3c99039df188440a](#)

Code In Scope

- [MorphoVaultV1Adapter.sol](#)
- src/AssetSilo.sol
- src/CollateralManager.sol
- src/Controller.sol
- src/RevenueModule.sol
- src/StakedAsset.sol
- src/external/chainlink/SpokeERC20.sol
- src/external/chainlink/SpokeERC20Restricted.sol
- src/oracle/GoldOracleAdapter.sol
- src/interface/IBurnMintERC20.sol
- src/interface/ICollateralManager.sol
- src/interface/IController.sol
- src/interface/IOracleAdapter.sol
- src/interface/IRestrictedRegistry.sol
- src/interface/IRevenueModule.sol
- src/interface/IStakedAsset.sol

Contact

Tenbin Labs CEO yuki@tenbinlabs.xyz

Tenbin Labs CTO jake@tenbinlabs.xyz

Commit hash: [820e700943c2b657c54219ab3c99039df188440a](#)

Audit Objectives

Goals

- Identify critical vulnerabilities
- Ensure compliance with best practice
- Analyze economic design
- Review privileged roles

High Surface Contracts

- Controller.sol
- CollateralManager.sol
- StakedAsset.sol
- [MorphoVaultV1Adapter.sol](#) (external)

Assumptions

- Only USD stablecoins will be used for collateral
- StakedAsset contract will have an initial deposit of 1e18
- OracleAdapter may use an oracle other than Chainlink
- Vaults in CollateralManager will be Morpho V2 vaults
- Morpho V2 vaults will be configured so that the CollateralManager is the only depositor and no performance/management fees
- For allocating to generic ERC4626 vaults, we will use the [MorphoVaultV1Adapter](#)
- SpokeERC20.sol will be used for CCIP on spoke chains for AssetToken and SpokeERC20Restricted will be used for StakedAsset tokens on spoke chains.
- Controller is agnostic to pricing provided in order data

External Dependencies

- [Morpho V2 Vault](#)
- [MorphoVaultV1Adapter](#)
- [Chainlink CCIP](#)
- [1Inch Router V6](#)
- [Chainlink XAU/USD Price Feed](#)

Areas of Concern

- StakedAsset
 - Need to ensure ID based cooldown system is functional
 - Instant unstaking and staked asset redemption (Controller) is a new feature which is previously not audited
- OracleAdapter & Controller Oracle Logic
 - Logic is somewhat complex and could be prone to errors
 - For redeeming staked assets, the logic uses share price as part of the final price calculation
- [MorphoVaultV1Adapter](#)
 - Need a guarantee that this contract can support allocating collateral from Morpho V2 Vault to arbitrary ERC4626 contracts. These adapters will hold a large portion of TVL in the Tenbin protocol.
- CollateralManager
 - The Controller has permission to call deposit() and withdraw() in the collateral manager, which directly links order execution with vault curation.
 - share_price is passed in via context in Controller
 - Ensure minimal slippage risk when interacting with SwapModule.
- Controller
 - For orders where a non-minter is executing an order, need to ensure there is minimal MEV risk when performing curation.
 - OracleAdapter interactions and calculations are critical
 - Mint and redemption limits are a previously unaudited feature

Protocol Overview

Tenbin is an asset token issuance platform with the goal of creating liquid, composable financial assets. Assets in the Tenbin protocol are backed by two positions: off-chain futures contracts and on-chain collateral. The off-chain hedging system maintains a delta one exposure of an underlying asset. The on-chain collateral is used to earn low-risk yield. So long as the on-chain yield equals or exceeds the off-chain funding costs, the protocol is able to peg Tenbin assets to the spot price of the real asset.

Each asset in the Tenbin protocol has a set of contracts unique to that asset. Key contracts include the AssetToken, Controller, StakedAsset, and CollateralManager. This document is an outline of the smart contract system used to facilitate mints, redemptions, staking, and on-chain collateral management.

AssetToken

An asset token represents an asset in the Tenbin protocol. The AssetToken contract is an immutable, non-upgradeable ERC20 token with an extension to allow minting and burning. There is an owner and minter role. The owner is a multisig that can set the minter account, and the minter can mint new tokens. In all cases, the minter will be set as the Controller contract.

Controller

The controller contract is responsible for minting and redeeming assets. Mint and redemptions are encoded as an Order. Orders are signed by KYC-approved signers and specify order details such as collateral amount, asset amount, and deadline. To successfully execute an order, a minter account calls the mint or redeem with an order and signature. Orders are executed atomically: collateral is transferred and tokens are minted/burned in a single transaction. All orders are executed by a minter key stored in a hardware security module and controlled by the Tenbin backend.

Order Fields

```
struct Order {
    OrderType order_type;        // Order type (MINT or REDEEM)
    uint256 nonce;              // Payer unique nonce
    uint256 expiry;             // Order expiration timestamp
    address payer;              // Account to transfer tokens from
    address recipient;          // Account to receive tokens
    address collateral_token;    // Collateral used for this order
    uint256 collateral_amount;   // Amount of collateral tokens
    address asset_token;        // Specify AssetToken or StakedAsset
    uint256 asset_amount;       // Amount of asset tokens
}
```

Order Lifecycle

1. KYC order signer is added to the allowed signers list
2. An order signer submits an order and signature to the back end
3. The back end processes the order
4. A context and approval are provided to the user to execute
5. Order is processed on chain and marked as completed

Allowed Signers

The controller keeps track of what accounts are allowed to sign orders. An account must be on the allowed signer list in order to submit a successful order. Note that the minter account can never modify the contents of an order, and that an order can only ever be executed once by specifying a unique nonce for every order. The controller supports both EIP712 and EIP1271 signatures.

Approved Recipients

Signers can set which accounts can receive tokens when an order is executed. This control prevents signers from sending tokens to an incorrect account. Signers can manage recipients on-chain by calling `setRecipientStatus()`. By default, a signer is approved to be a recipient for its own orders.

Delegate Signers

Any account can delegate a signer to sign orders on its behalf. During execution of an order, the payer is checked against the delegates for a signer. This allows for EOAs or smart contracts to let a signer sign an order where the delegate pays for the order. An account can add a delegate signer by calling `setDelegateStatus()`.

Collateral Ratio

The controller has a collateral ratio which specifies the percentage of collateral which is sent to `custodian` and `manager`. The custodian amount represents the portion of collateral which is moved off-chain to fund futures hedge positions. The manager amount represents the portion of collateral designated to earn on-chain yield. When a mint occurs, the total collateral amount is split according to the `ratio` value set in the Controller.

Custodian Module

The controller has a collateral ratio which specifies the percentage of collateral which is sent to custodian and manager. The custodian amount represents the portion of collateral which is moved off-chain to fund futures hedge positions. The manager amount represents the portion of collateral designated to earn on-chain yield. When a mint occurs, the total collateral amount is split according to the ratio value set in the Controller.

Oracle Adapter

The controller has a configurable oracle adapter which can provide a price when executing orders. When enabled, the oracle price acts as a backstop to prevent order pricing from exceeding a threshold. The oracle DOES NOT determine the price of assets, rather it acts as a security measure to prevent minting and redeeming assets at a price off-peg. For example, with the oracle configured it is impossible to mint new assets with a price of \$0.

Restricted Registry

The controller uses a restricted registry of accounts which cannot interact with the controller. During order execution, the `payer` and `recipient` are checked against this registry.

Mint and Redemption Limits

Mint and redemption limits are configurable per block. If set to `type(uint).max`, limits are not enforced.

Staked Asset Redemption

Staked assets can be redeemed directly through the controller by specifying `asset_token` in the `Order` struct. Redeeming staked assets also requires the payer approves the controller to spend staked assets on its behalf. During staked asset redemption, assets are instantly unstaked by the controller and pricing is determined by the back end pricing engine.

CollateralManager

The manager contract stores collateral to earn on-chain yield and provide liquidity for redemptions. When executing a mint order, collateral is transferred directly to the `CollateralManager`. The manager is non-custodial: there is no way to withdraw protocol collateral from the manager except during execution of a redemption order or a rebalancing action. For each collateral there is an associated ERC4626 vault, typically a Morpho V2 vault, used to earn yield. The manager uses accounting to separate collateral from revenue.

The main actions that can be performed by the collateral manager are:

- Deposit collateral into vaults
- Withdraw collateral from vaults
- Swap collateral tokens
- Withdraw revenue
- Rebalance collateral between on/off chain

Deposit and Withdraw

The two main functions provided by the collateral manager are ``deposit()`` and ``withdraw()``. Each collateral has a unique vault dedicated to earning yield for that collateral type. Depositing will deposit collateral into the vault in exchange for vault shares. Withdrawing will redeem vault shares in exchange for the underlying collateral.

Liquidity Management

The most important role of the manager is to ensure there is sufficient liquidity to redeem assets. Collateral that is in the collateral manager is approved to be transferred by the controller when a redemption order is filled. If there is insufficient collateral in the manager, a redemption is not possible.

In order for Tenbin to optimize yield versus liquidity, a curator role is assigned by the manager to perform deposits and withdrawals. By using a multicall contract, it is possible to bundle deposits and withdrawals with mints and redemptions atomically. For example: ``[mint(), deposit()]`` or ``[withdraw(), redeem()]``.

Collateral Rebalancing

Rebalancing occurs when there is a surplus or deficit of on-chain vs off-chain collateral. If the off-chain position is over-collateralized, collateral is moved on-chain through a custodian account and transferred directly to the Collateral Manager. If the off-chain position is under-collateralized, the ``rebalancerWithdraw()`` function can be called by a permissioned role to withdraw collateral from the manager.

In order to limit the amount of collateral that can be withdrawn during a rebalance, there is a cap set on the amounts that can be withdrawn during this function. Additionally, the rebalancer withdraw can only withdraw to a registered set of custodians in the CustodianModule. This design guarantees that protocol collateral is always in custody and cannot be directly controlled by protocol operators.

Revenue Module

The RevenueModule manages revenue earned by the protocol. In most cases, revenue is transferred back to the collateral manager in order to pay for the off-chain hedging costs.

The revenue module can perform the following actions:

- Withdraw revenue from the collateral manager
- Transfer revenue back to the collateral manager
- Transfer revenue to a multisig contract
- Provide liquidity to mint new asset tokens as a reward
- Reward the staking pool with asset tokens

A keeper role is assigned by the revenue module to automate these tasks. For example, the keeper might be called 2x per day to transfer revenue back to the collateral manager, and 1x per day to reward the staking contract.

Swap Module

The swap module is used to perform on-chain swaps between collateral types. The goal of this module is to strictly restrict what types of swaps are possible and limit the slippage between different collaterals. Each swap performed in the CollateralManager passes in a set of constraints for that swap plus the call data to perform the swap. In addition, there are configurable slippage limits in the contract storage for swaps between specific collaterals. This security measure limits the possibility of performing a poorly priced swap. Additionally, there is a swap capacity for each token. The swap cap prevents the curator from swapping an excessive amount of collateral (for instance, human or backend error)

It is possible to bundle swaps as part of a multicall. For example, if there is insufficient USDT in the manager and a redemption order is placed requesting USDT, the following bundle can be created: `[swap(), redeem()]`.

Staked Asset

The staking contract allows accounts to stake asset tokens in exchange for a staking token. If the protocol is profitable, revenue can be used to mint new asset tokens and reward them to the staking pool. Staking allows for the creation of compounding, yield-bearing assets in the Tenbin protocol. The staking contract is implemented as a custom ERC4626 vault.

Locking assets in the staking contract has the added benefit of enabling advanced yield strategies in the manager vaults. For example, if it is known that 20% of assets are locked in the staking pool for 7 days, the portion of collateral backing those assets can be committed to locked yield strategies such as sUSDe. Additionally, understanding how much of an asset is locked allows more efficient liquidity management in the manager.

The asset value of a staking token can only increase over time. When unstaking, the staking token is burned in exchange for the original amount deposited, plus a share of any rewards earned since staking.

Vesting

A vesting period is used for rewards in order to prevent abuse of the staking contract. The vesting period encourages depositors to remain staked through the vesting period. This mechanism prevents negative MEV actions such as sandwiching reward transactions.

Additionally vesting spreads out rewards over a longer period of time in order to reduce reward volatility. For some assets, minting reward tokens is not always possible due to futures off-market hours. Using a longer vesting period allows stakers to earn a consistent yield despite possibly inconsistent reward schedules and amounts.

Cooldown

A cooldown period is present in the staking contract to encourage assets to remain staked and allow response time for liquidity management. When a staker calls the `cooldownShares()` or `cooldownAssets` function, staked tokens are burned and transferred to the `AssetSilo`

contract. An account can have multiple cooldowns at once as determined by a unique set of IDs per account. After the cooldown end time has passed, a user can call `unstake(id)` to withdraw the underlying asset tokens.

In order to withdraw tokens, a staker needs to call `cooldown()` and wait until the cooldown period has passed to withdraw their stake. A cooldown can be cancelled by calling `cancelCooldown(id)`. When cancelling a cooldown, new shares are minted via the `AssetSilo` contract and transferred to the account which cancelled the cooldown.

Each staker can only have one cooldown at a time, and the cooldown will be reset when cooling down additional assets. Initiating a cooldown cannot be cancelled - new cooled down amounts will be added to the previous cooldown amount. The cooldown period is designed to improve collateral management and prevent abuse of the staking contracts.

Instant Unstaking

Instant unstaking is possible via a privileged role. Typically, the Controller is allowed to instant unstake when executing redemptions for staked assets. There is an `instantUnstakeCap` which sets a limit on the total amount that can be instantly unstaked. An optional instant unstaking fee which, when enabled, will transfer a percentage of assets to a fee recipient. This fee is disabled by default.

Restricted Registry

Due to legal restrictions, yield cannot be paid to stakers without regulatory compliance. For this reason, a restricted registry is present in the staking contract. Accounts added to this registry cannot stake, unstake, or transfer staked tokens. If an account is restricted, the contract default admin can burn the account's staking tokens and withdraw the underlying assets.

Roles

A variety of roles are used in the Tenbin contracts to achieve a separation of concerns and granular level of control.

Role	Security	Description
DEFAULT_ADMIN_ROLE	Owner Multisig (5/7)	Grant and revoke roles Set <code>minter</code> in AssetToken Set manager address in controller Set custodian address in controller Upgrade CollateralManager Upgrade StakedAsset Add and remove collateral types Add and remove custodian accounts Set swap module in CollateralManager Set controller in CollateralManager Set revenueModule in CollateralManager
ADMIN_ROLE	Admin Multisig (5/7)	Adjust controller ratio Set oracle adapter and oracle delta tolerance Rescue tokens and ether stuck in contracts Change vesting period in staking contract Change cooldown period in staking contract

MINTER_ROLE	HSM #1 MultiCall	Execute valid mint and redemption orders
MULTICALLER_ROLE	HSM #1	Call Multicall contract
CURATOR_ROLE	HSM #2 MultiCall Controller	Call <code>deposit()</code> in CollateralManager Call <code>withdraw()</code> in CollateralManager Call <code>swap()</code> in CollateralManager
REBALANCER_ROLE	HSM #2	Withdraw to custodian module
CUSTODIAN_KEEPER_ROLE	HSM #2	Automate transfers to custodians
REVENUE_KEEPER_ROLE	HSM #2 Tenbin Multisig (4/7)	Pull revenue into RevenueModule Transfer revenue to CollateralManager Withdraw revenue to Multisig Approve the Controller to mint AssetTokens Reward staking contract with Asset Tokens
GATEKEEPER_ROLE	HSM #3 Owner Multisig (5/7)	Pause mint and redemption in controller Emergency pause Controller Emergency pause CollateralManager
REWARDER_ROLE	Tenbin Multisig (4/7) RevenueModule	Send rewards to the StakedAsset contract

RESTRICTER_ROLE	Admin Multisig (5/7)	Restrict accounts in Controller Restrict accounts in StakedAsset
SIGNER_MANAGER_ROLE	Developer Multisig (3/7)	Add and remove signers in controller
CAP_ADJUSTER_ROLE	Developer Multisig (3/7)	Set rebalance caps in CollateralManager Set swap caps in CollateralManager Change min swap prices in CollateralManager
INSTANT_UNSTAKER_ROLE	Controller	Call instantUnstake() function in StakedAsset

Parameters

Parameter	Contract	Purpose
minter	AssetToken	Account which can mint new asset tokens
pauseStatus	Controller	None $\Rightarrow$ Not Paused MintRedeemPause $\Rightarrow$ Pause mint & redeem functions FMLPause $\Rightarrow$ Emergency Pause
isCollateral	Controller	true if a token is collateral in the Controller
ratio	Controller	Percentage of collateral to transfer to custodian
oracle.adapter	Controller	Address of price oracle
oracle.tolerance	Controller	Percentage tolerance between oracle price and order price. 1e18 = 100%
isRestricted	Controller	Add and remove restricted accounts. Restricted accounts cannot act as the payer or recipient during order execution.
controller	CollateralManager	Address of controller contract. Changed if a new controller is created
swapModule	CollateralManager	Address of swap module to perform swaps
pauseStatus	CollateralManager	None $\Rightarrow$ Not Paused FMLPause $\Rightarrow$ Emergency Pause

vaults	CollateralManager	Add ERC4626 vault for collaterals
rebalanceCap	CollateralManager	Maximum amount of a collateral the rebalancer can withdraw. When <code>rebalance()</code> is called the amount withdrawn is subtracted from the rebalance cap for that collateral
swapCap	CollateralManager	Set the swap cap for a specific token. When swapping this token out, the cap is decreased
minSwapPrice	CollateralManager	Set the minimum amount of tokens received per 1 token in
custodians	CustodianModule	Approved custodian accounts which can receive collateral
cooldownPeriod	StakedAsset	Set cooldown period for unstaking in seconds
isRestricted	StakedAsset	Add and remove restricted accounts. Restricted accounts cannot stake, unstake, or transfer staked assets. Restricted accounts can have their balance withdrawn by an admin account
vesting.period	StakedAsset	Set the period for which rewards vest to the staking contract in seconds

END